UNIVERSIDAD
Panamericana

PROGRAMACIÓN AVANZADA

Sesión 11

Funciones Recursivas

✎ NOTAS DEL PROFESOR

Universidad Panamericana
Dr. Miguel Alcaraz Rivera • malcaraz@up.edu.mx

2026

Índice

Estructura de la sesión

Tiempo	Bloque	Tipo
10 min	Historia: Fibonacci, Gödel, LISP	Exposición
15 min	Definición, partes, primer ejemplo (`suma`)	Exposición + pizarrón
15 min	El call stack: diagrama en vivo	Demo en cuaderno
10 min	Stack overflow: `f2.cpp` en pizarrón	Demostración
15 min	Factorial, potencias, Fibonacci	Ejemplos guiados
15 min	Ejercicios 1–4 en parejas	Trabajo en clase
80 min		

Preguntas de arranque

Antes de abrir el cuaderno o el pizarrón, lanza estas preguntas al grupo:

1. **"¿Qué pasa si defines una palabra del diccionario usando esa misma palabra en la definición?"**
   Respuesta esperada: es circular, no ayuda a entender. Llevar esto a: una función recursiva bien hecha *no* es circular porque cada llamada usa un problema estrictamente más pequeño.

2. **"¿Cuánto es la suma de los primeros 100 enteros? ¿Y si no puedes usar la fórmula $n(n+1)/2$?"**
   Respuesta esperada: sumar uno por uno. Entonces: ¿y si usas que suma$(100) =$ $100 +$ suma(99)? Esto prepara el terreno para la definición recursiva.

3. **"¿Alguien conoce las Torres de Hanoi?"**
   Sin entrar en detalle — solo sembrar curiosidad para la sesión 12.

Notas históricas ampliadas

La paradoja del diccionario (Hofstadter). En su libro *Gödel, Escher, Bach* (1979, Premio Pulitzer), **Douglas Hofstadter** dedicó extensos capítulos a la recursión como concepto unificador entre las matemáticas, la música de Bach y el arte de Escher. La idea de los "bucles extraños" (*strange loops*) — sistemas que se referencian a sí mismos — es central en el libro. Vale la pena mencionarlo como lectura opcional para alumnos interesados; es uno de los libros más influyentes en la intersección de matemáticas, filosofía y cómputo.
Referencia: Hofstadter, D. *Gödel, Escher, Bach: An Eternal Golden Braid*. Basic Books, 1979.

Las siglas de GNU. El proyecto GNU ("GNU's Not Unix") tiene un nombre que es en sí mismo recursivo: GNU se define usando GNU. Esto es un guiño explícito de Richard Stallman a la tradición de los hackers de los 80. Es un

ejemplo moderno y cotidiano de recursión en el lenguaje natural.

¿Por qué FORTRAN no podía hacer recursión? FORTRAN (1957) fue diseñado con la asunción de que cada función tenía una única copia de sus variables locales — almacenadas en posiciones fijas de memoria. Si una función se llamaba a sí misma, la segunda llamada sobreescribía los datos de la primera. La solución fue agregar una *pila de llamadas* dinámica, idea que se atribuye principalmente al equipo de ALGOL (1960) y que LISP implementó de forma elegante en la misma época. La pila de llamadas es tan fundamental hoy que es difícil imaginar un procesador sin ella.

Mutua recursión. Dos funciones pueden ser mutuamente recursivas: f llama a g y g llama a f. Un ejemplo clásico es la comprobación de paridad: `esImpar(n) = esPar(n-1)` y `esPar(n) = esImpar(n-1)`, con casos base `esPar(0) = true` y `esImpar(0) = false`. Mencionarlo como curiosidad si el tiempo lo permite.

Demo recomendada: visualizar el call stack en vivo

La celda de `sumaTraza` del cuaderno muestra la indentación proporcional al nivel de profundidad. Antes de ejecutarla, dibuja en el pizarrón la pila de frames manualmente para `suma(3)`:

1. Dibuja un rectángulo para `main()` al fondo.
2. Encima, dibuja `suma(3)` con su parámetro `x=3`. Escribe "espera..." porque aún no puede calcular el resultado.
3. Sigue apilando `suma(2)`, `suma(1)`, `suma(0)`.
4. Al llegar al caso base, tacha el frame de `suma(0)` y escribe "retorna 0" con una flecha hacia abajo.
5. Ve sustituyendo valores hacia abajo en la pila.

Luego ejecuta `sumaTraza(4)` en el cuaderno. Los alumnos van a ver que la indentación replica exactamente el diagrama del pizarrón. El impacto visual de ver el código confirmar el dibujo es alto.

FAQs: dudas frecuentes con respuestas

A veces sí, a veces no. Para `suma` o `factorial`, el ciclo es marginalmente más eficiente porque no crea frames extra. Pero para problemas como las Torres de Hanoi, recorridos de árboles o algoritmos de backtracking, la solución recursiva es órdenes de magnitud más legible y la diferencia de rendimiento es irrelevante o inexistente. El mensaje clave: **elige la herramienta que expresa mejor la estructura del problema**.

Depende del sistema y del tamaño de cada frame. En la práctica: con `suma(n)` en Linux x86-64, el límite está alrededor de $n \approx 100{,}000$–$300{,}000$. El ejercicio 7 de las notas pide calcularlo teóricamente: 8 MB$/32$ bytes/frame $= 262{,}144$ frames. Ese número es una estimación; el frame real incluye más datos (dirección de retorno, registros guardados, etc.), así que el límite real es menor.

Si un alumno prueba en su máquina y el resultado difiere, está bien: eso ilustra que el límite es específico del sistema.

Ambos funcionan como caso base: con $0! = 1$ el programa llega hasta $n = 0$, con $1! = 1$ se detiene antes. Sin embargo, $0! = 1$ es la definición **matemáticamente correcta** del factorial (la combinatoria requiere que el número de formas de ordenar cero elementos sea 1). Usar $1! = 1$ como caso base es válido en código pero matemáticamente incompleto porque `factorial(0)` no estaría definido.

Computacionalmente, sí — cualquier función recursiva puede transformarse en un ciclo y viceversa (son equivalentes en expresividad). Conceptualmente, no: la recursión expresa “el problema se define en términos de sí mismo”, lo que a veces refleja la estructura natural del problema mucho mejor que un ciclo. La elección es de estilo y claridad, no de capacidad.

Memoización es guardar los resultados de llamadas anteriores para no recalcularlos. Con Fibonacci, si guardamos $F(3)$ la primera vez que lo calculamos, las siguientes llamadas a $F(3)$ simplemente leen ese valor. El tiempo pasa de $O(2^n)$ a $O(n)$. Es un anticipo de *programación dinámica*, que se ve en cursos de algoritmos. No es necesario profundizar en esta sesión; basta con mencionar que el problema existe y tiene solución elegante.

Soluciones a los ejercicios

$$\begin{aligned}
\texttt{suma(6)} &= 6 + \texttt{suma(5)} = 6 + 15 = 21 \\
\texttt{suma(5)} &= 5 + \texttt{suma(4)} = 5 + 10 = 15 \\
\texttt{suma(4)} &= 4 + \texttt{suma(3)} = 4 + 6 = 10 \\
\texttt{suma(3)} &= 3 + \texttt{suma(2)} = 3 + 3 = 6 \\
\texttt{suma(2)} &= 2 + \texttt{suma(1)} = 2 + 1 = 3 \\
\texttt{suma(1)} &= 1 + \texttt{suma(0)} = 1 + 0 = 1 \\
\texttt{suma(0)} &= 0 \quad \text{(caso base)}
\end{aligned}$$

Resultado: `suma(6)` $= 21$. Frames apilados simultáneamente: **8** (`main` + `suma(6)` + ... + `suma(0)`).

$6! = 6 \times 5 \times 4 \times 3 \times 2 \times 1 \times 1 = 720$.
Multiplicaciones realizadas: **6** (una por cada llamada recursiva, excluyendo el caso base que solo retorna 1).

`pot(3,4)` $= 3\times$ `pot(3,3)` $= 3\times3\times$ `pot(3,2)` $= 3\times3\times3\times$ `pot(3,1)` $= 3\times3\times3\times3\times$ `pot(3,0)` $= 3^4 = 81$. ✓

Problema: el caso recursivo llama con `n + 1` en lugar de `n - 1`. El argumento *crece* en lugar de decrecer, así que nunca se acerca a ningún caso base. Si hubiera un caso base para $n = 0$, una llamada con $n > 0$ **nunca** lo alcanzaría. Resultado: recursión infinita $\rightarrow$ stack overflow.

Versión corregida (suponiendo que la intención era sumar de `n` hasta 0):

```
int buena(int n) { if (n==0) return 0; return n + buena(n-1); }
```

a) Suma de los primeros n impares:

$$S(1) = 1, \qquad S(n) = (2n - 1) + S(n - 1)$$

Curiosidad: $S(n) = n^2$ siempre. Pídeles que lo verifiquen con $n = 5$.

b) Producto $n \cdots m$:

$$P(n, n) = n, \qquad P(n, m) = n \times P(n + 1, m) \quad (n < m)$$

Alternativa con $n > m$ como caso base:

$$P(n, m) = 1 \ \text{ si } n > m, \qquad P(n, m) = n \times P(n + 1, m)$$

Árbol simplificado (se omiten hojas triviales):

$$F(7) = F(6) + F(5) = 8 + 5 = 13$$

Los primeros valores: $1, 1, 2, 3, 5, 8, 13$. ✓

$F(3)$ aparece en el árbol de $F(7)$: lo calcula $F(5)$ que lo necesita, $F(4)$ que lo necesita, y sub-llamadas de cada uno. En total, $F(3)$ se calcula **5 veces** al calcular $F(7)$.

En general, el número de llamadas para calcular $F(n)$ es aproximadamente $2^{n/2}$ — crecimiento exponencial. Para $F(40)$ son más de 10^8 llamadas.

8 MB $= 8 \times 1{,}048{,}576 = 8{,}388{,}608$ bytes.
Con frames de 32 bytes: $8{,}388{,}608/32 = 262{,}144$ frames.
Por tanto, la profundidad teórica máxima es $\approx 262{,}144$, lo que equivale a llamar `suma(262143)`.

En la práctica el frame es mayor (en Linux x86-64 con `g++` suele ser 32–64 bytes con optimizaciones desactivadas), así que el límite real está entre 100,000 y 250,000. Lo interesante pedagógicamente es que el límite *existe* y es calculable.

Conexión con las sesiones 12 y 13

Al final de la sesión, si el tiempo lo permite, menciona que la **sesión 12** tendrá una actividad de diseño: implementar las Torres de Hanoi como arte ASCII en equipos de tres. No des la solución del algoritmo recursivo todavía — es importante que experimenten con el diseño visual primero y que lleguen a la sesión 13 con el código de arte ya hecho.

El algoritmo para mover n discos de la torre A a la C usando B como auxiliar es:

1. Mueve los $n - 1$ discos superiores de A a B (usando C como auxiliar).
2. Mueve el disco más grande de A a C.
3. Mueve los $n - 1$ discos de B a C (usando A como auxiliar).

El caso base es $n = 0$: no hay nada que mover. El número de movimientos es $2^n - 1$ — verificable con la traza. **No anticipes esto** en la sesión 11; déjalo como descubrimiento de la sesión 13.